Cyber Security

SIS: Firewall/Hash/Port Scanner

Fullname: Amangeldi Zhanserik
ID: 22B030301
E-mail: zha_amangeldi@kbtu.kz
Date of submission: March 27, 2025
Class time: Thursday, 15:00–18:00

School of Information System and Engineering
Kazakh-British Technical University
Academic Year 2024-2025

Contents

1 Lab 1: Configuring a Firewall

1.1 Objective

In this lab, I worked with Linux firewall configurations using iptables. The main objectives were:

- Installing and configuring an Apache web server
- Managing firewall rules with iptables to control HTTP traffic
- Analyzing firewall logs to understand connection attempts

1.2 Key Steps and Observations

1.2.1 Installing and Starting Apache Server

I installed Apache on the Security Workstation using the following commands:

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl start apache2
sudo systemctl enable apache2
```

After installation, I verified the server was active by checking its status:

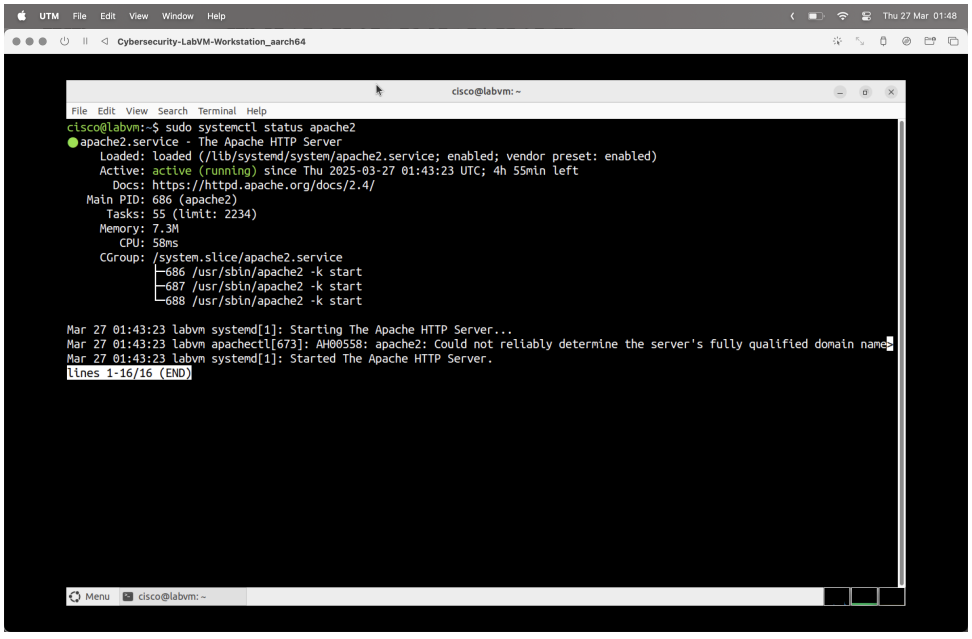

Figure 1: Apache server status showing active

The IP address of my VM was determined to be: **172.18.1.45**

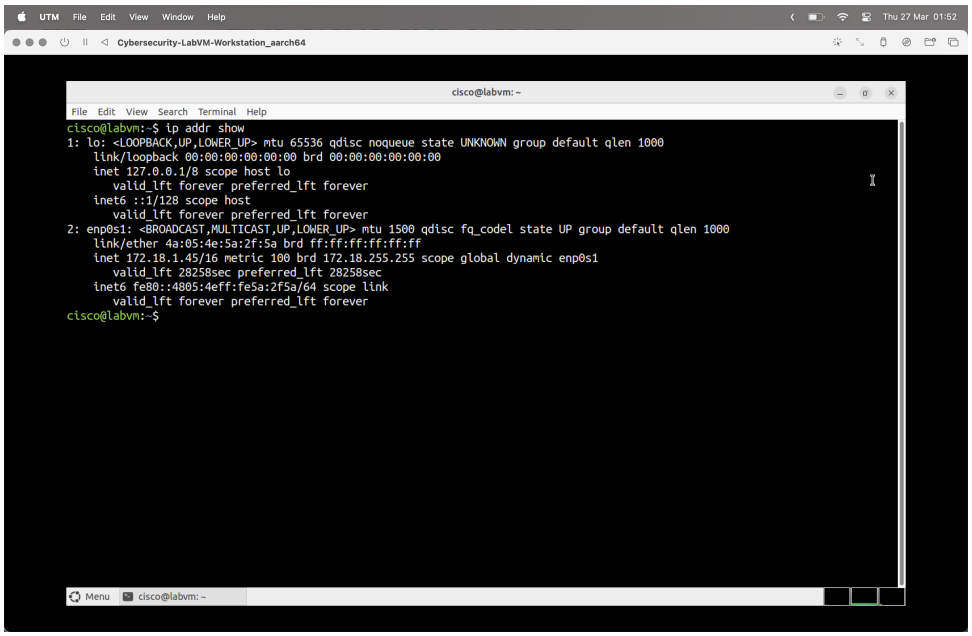

Figure 2: Checking the ip of VM using command **ip addr show**

1.2.2 Testing Web Server Access

I confirmed that the Apache server was accessible from my host machine by navigating to the VM's IP address in a browser:

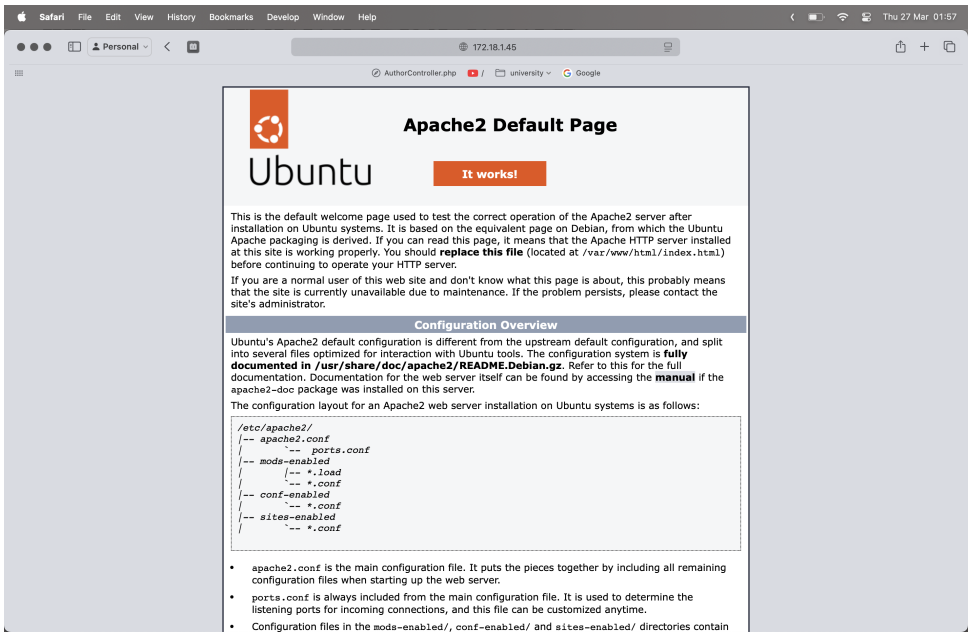

Figure 3: Default Apache page displayed in browser

1.2.3 Configuring Firewall Rules

I implemented the following iptables rules:

1. First blocked HTTP traffic:

```
sudo iptables -I INPUT 1 -p tcp --destination-port 80 -j
    ↪ DROP
```

2. Verified the block was effective by attempting to access the server again:

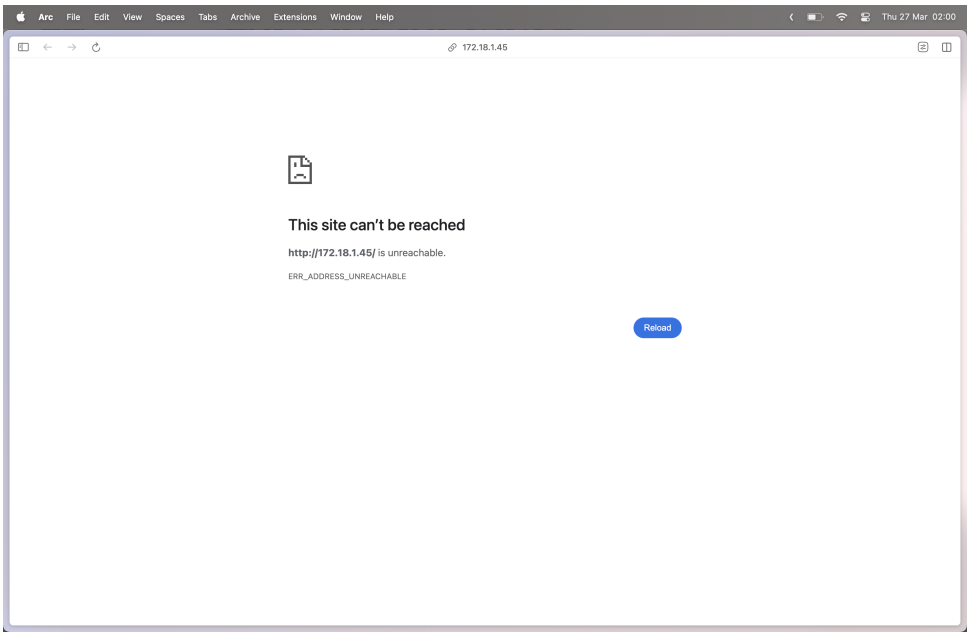

Figure 4: Connection failed after blocking port 80

3. Re-enabled HTTP access:

```
sudo iptables -I INPUT 1 -p tcp --destination-port 80 -j
    ↪ ACCEPT
```

4. Added logging to monitor traffic:

```
sudo iptables -I INPUT 1 -j LOG
```

The final rule configuration looked like this:

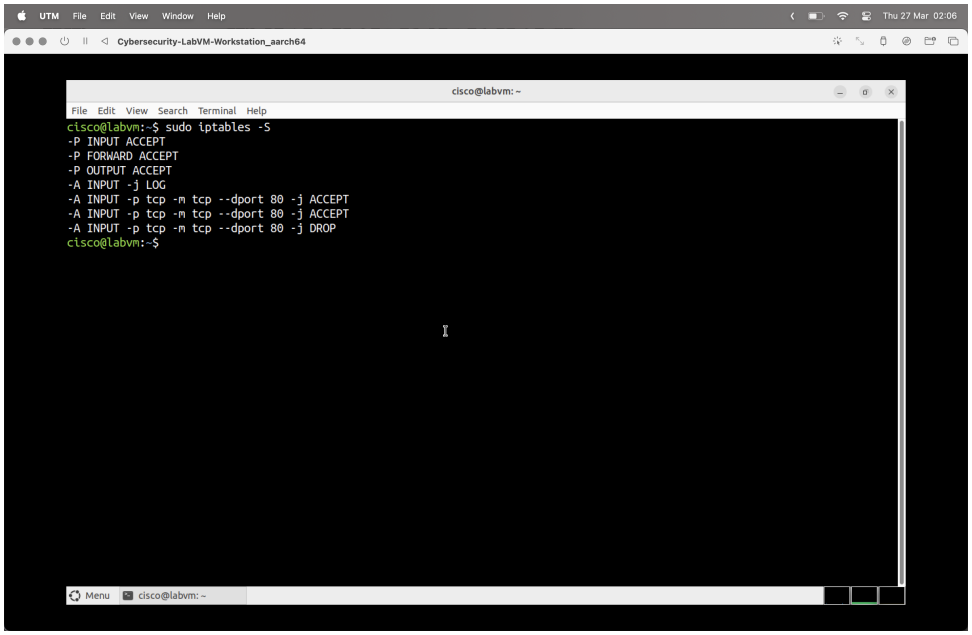

Figure 5: Final iptables rule configuration

1.2.4 Analyzing Firewall Logs

I examined the logs using the following command:

```
sudo tail /var/log/kern.log | grep "80"
```

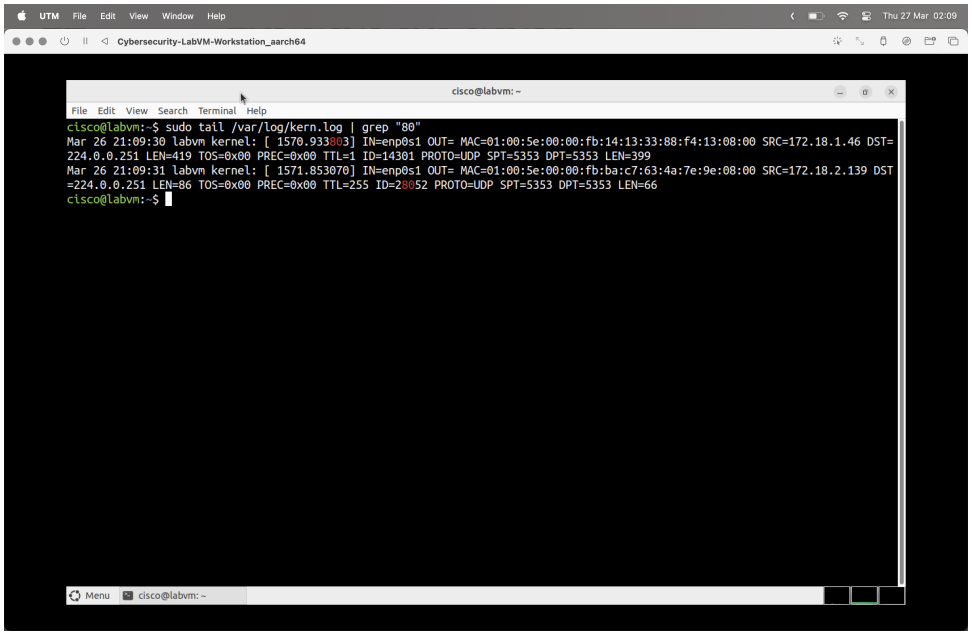

Figure 6: Firewall Logs

From the logs, I observed:

- The source IP addresses attempting to connect to the server
- The packet types (SYN, ACK, etc.) being exchanged

- Connection attempts with their timestamps
- Request frequencies and patterns

1.3 Lab 1 Conclusion

Through this lab, I learned how to:

- Configure Linux firewall rules using iptables
- Understand the relationship between firewall rules and network services
- Monitor and interpret network traffic using system logs
- Apply rule ordering to achieve desired network security policies

This exercise demonstrated how firewalls can be used to protect servers by controlling access to specific ports, and highlighted the importance of monitoring network traffic for security purposes.

2 Lab 2: Using a Port Scanner to Detect Open Ports

2.1 Objective

The main objective of this lab was to use Nmap to scan for open ports on a system and analyze the security implications of these findings.

2.2 TCP Port Scanning Results

I ran a basic Nmap scan against localhost:

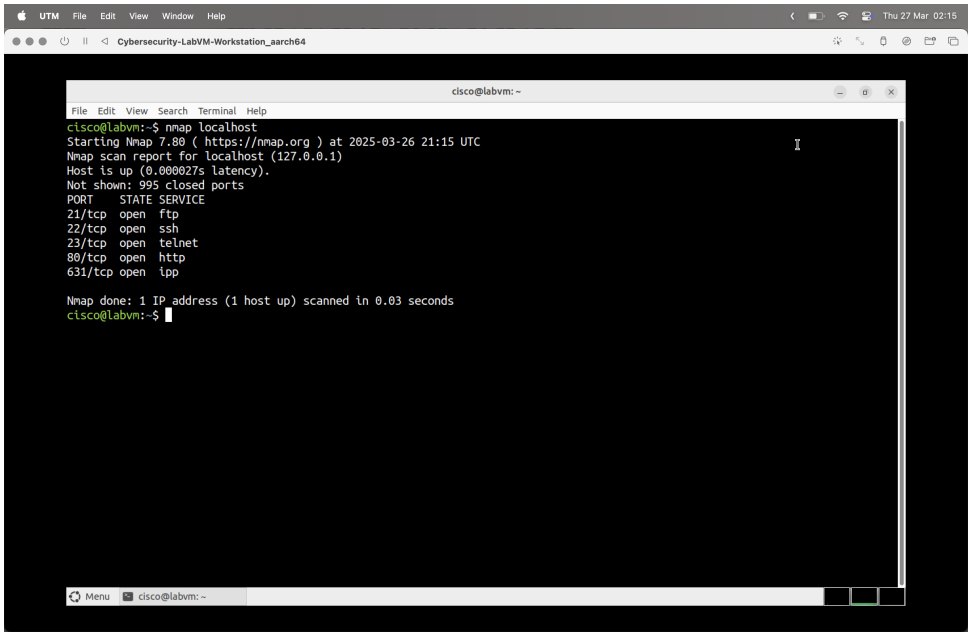

Figure 7: Basic Nmap scan results showing open TCP ports

Open TCP Ports Found:

- Port 21 (ftp)
- Port 22 (SSH)
- Port 23 (Telnet)
- Port 80 (http)
- Port 631 (IPP)

Service Analysis:

- **FTP (Port 21)** - File Transfer Protocol used for transferring files between systems. By default, it transmits credentials and data in plaintext, making it insecure without encryption (FTPS or SFTP).
- **SSH (Port 22)** - Secure Shell provides encrypted remote access and administration, replacing insecure alternatives like Telnet.

- **Telnet (Port 23)** - An unencrypted protocol for remote terminal access. It sends all data, including passwords, in plaintext, making it highly vulnerable.
- **HTTP (Port 80)** - Hypertext Transfer Protocol used for web communication. Without HTTPS (TLS encryption), traffic can be intercepted or modified.
- **IPP (Port 631)** - Internet Printing Protocol used by CUPS (Common Unix Printing System) for network printing services.

Vulnerability Analysis:

- **FTP**: Susceptible to brute-force attacks and MITM (Man-in-the-Middle) attacks. Unencrypted data transmission exposes credentials and files. Anonymous access misconfigurations can lead to data exposure.
- **SSH**: While secure, it is a common target for brute-force attacks if weak passwords are used. Older versions may contain vulnerabilities like CVE-2021-41617.
- **Telnet**: Completely insecure due to plaintext data transmission. Attackers can intercept credentials using packet sniffing. It should be replaced with SSH.
- **HTTP**: Vulnerable to MITM attacks if no SSL/TLS is used. Prone to SQL injection, Cross-Site Scripting (XSS), and other web-based exploits.
- **IPP**: The CUPS service on port 631 has had multiple reported vulnerabilities, including privilege escalation, denial-of-service (DoS) attacks, and unauthorized access to print jobs.

2.3 UDP Port Scanning Results

I scanned for open UDP ports using sudo privileges:

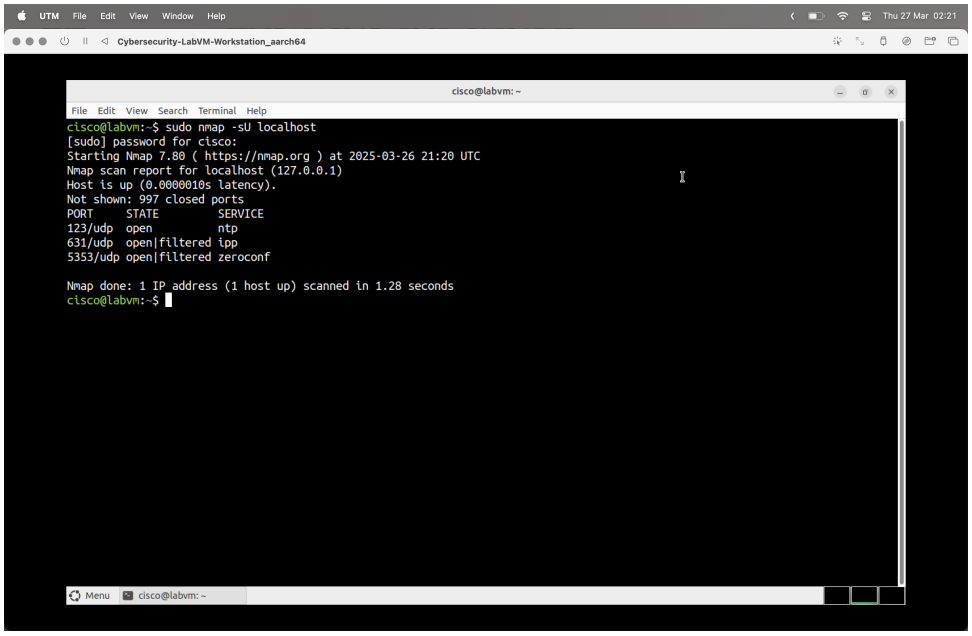

Figure 8: Nmap UDP scan results

Open UDP Ports Found:

- Port 123 (NTP)
- Port 631 (IPP)
- Port 5353 (Zeroconf/mDNS)

Service Analysis:

- **NTP (Port 123)** - Network Time Protocol is used to synchronize system clocks across a network. It ensures accurate timekeeping for security protocols and system logs.
- **IPP (Port 631)** - Internet Printing Protocol is used by CUPS (Common Unix Printing System) for network printing and printer management.
- **Zeroconf/mDNS (Port 5353)** - Multicast DNS (mDNS) is used for service discovery on local networks, allowing devices to find each other without a central DNS server.

Vulnerability Analysis:

- **NTP**: Vulnerable to amplification attacks, where an attacker spoofs the victim's IP address and floods it with large responses (NTP reflection attack). Older NTP versions may also be susceptible to DoS and time manipulation attacks.
- **IPP**: The CUPS service has had various security flaws, including privilege escalation and remote code execution vulnerabilities (e.g., CVE-2023-4504). Misconfigured printers may allow unauthorized access to print jobs.
- **Zeroconf/mDNS**: Vulnerable to information leakage and MITM (Man-in-the-Middle) attacks. Attackers can exploit mDNS to enumerate network services and potentially manipulate responses for phishing or network attacks.

2.4 Service Version Detection

I performed a service version scan to get more detailed information:

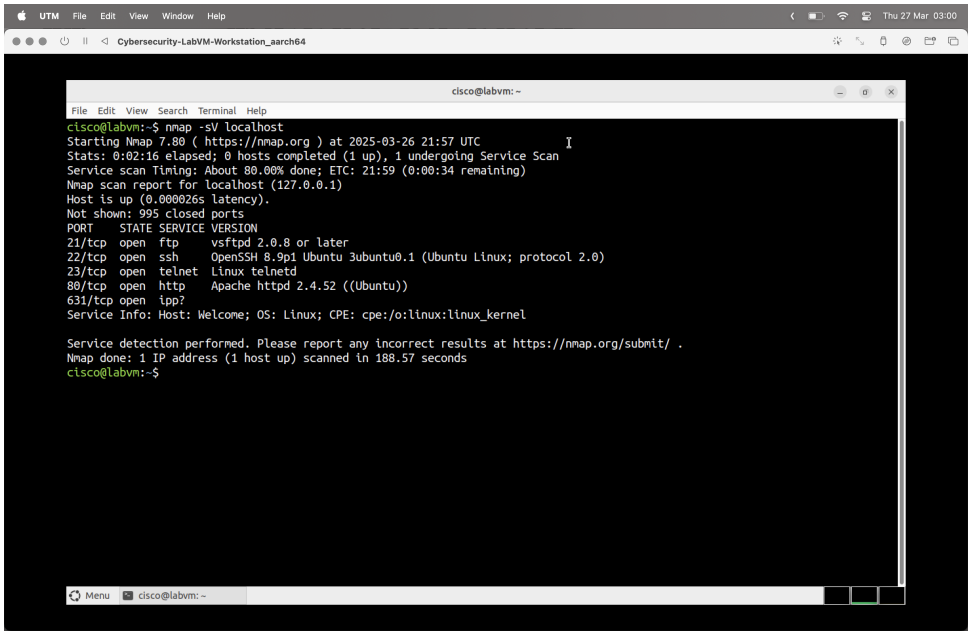

Figure 9: Nmap service version detection results

This scan revealed specific software versions:

- FTP: vsftpd 2.0.8 or later
- SSH: OpenSSH 8.9p1 Ubuntu 3ubuntu0.1
- Telnet: Linux telnetd
- HTTP: Apache httpd 2.4.52
- IPP: ?

These specific versions are important for vulnerability assessment, as each version may have particular known exploits.

2.5 SSH Key Information

Using the aggressive scanning mode, I obtained SSH host keys:

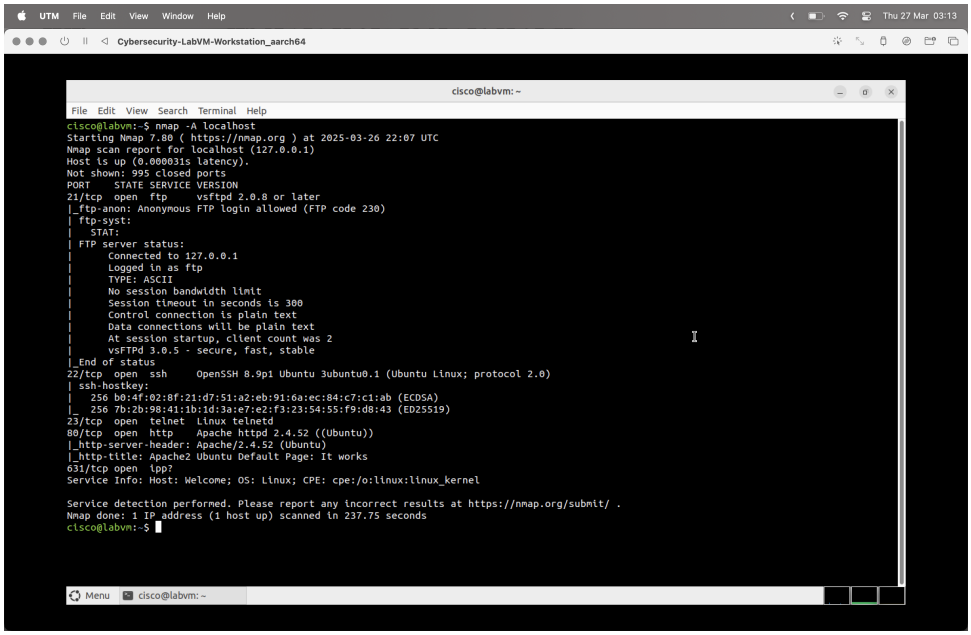

Figure 10: Nmap aggressive scan showing SSH host keys

Captured SSH Host Keys:

- ECDSA (256-bit): `b0:4f:02:8f:21:d7:51:a2:eb:91:6a:ec:84:c7:c1:ab`
- ED25519 (256-bit): `7b:2b:98:41:1b:1d:3a:e7:e2:f3:23:54:55:f9:d8:43`

How an Attacker Could Use This Information:

- Identify the system by its SSH fingerprint
- Detect if the server has been replaced (if the fingerprint changes)
- Perform SSH MITM attacks if hostkey verification is disabled

Mitigation Strategies:

- Disable unnecessary SSH key types (like DSA)
- Use SSH key rotation policies
- Implement firewall rules to limit SSH access to trusted IP addresses
- Use key-based authentication instead of passwords
- Configure SSH to use stronger ciphers and MAC algorithms

2.6 Lab 2 Conclusion

This port scanning exercise revealed several potential security issues:

- The presence of Telnet (port 23) poses a significant security risk
- Multiple services (SSH, IPP, Zeroconf) provide potential attack vectors
- Detailed system information is discoverable through simple scanning techniques

To improve security, I would recommend:

- Disabling Telnet entirely and using SSH exclusively
- Restricting access to necessary ports only
- Implementing proper authentication mechanisms
- Keeping all services updated to patch known vulnerabilities
- Configuring the firewall to limit access to these services

3 Lab 3: Hashing Things Out

3.1 Part 1: Hashing a Text File with OpenSSL

In this part of the lab, I explored hash functions by creating and comparing hashes of text files.

3.1.1 Initial Text File Hashing

I first examined the contents of the text file:

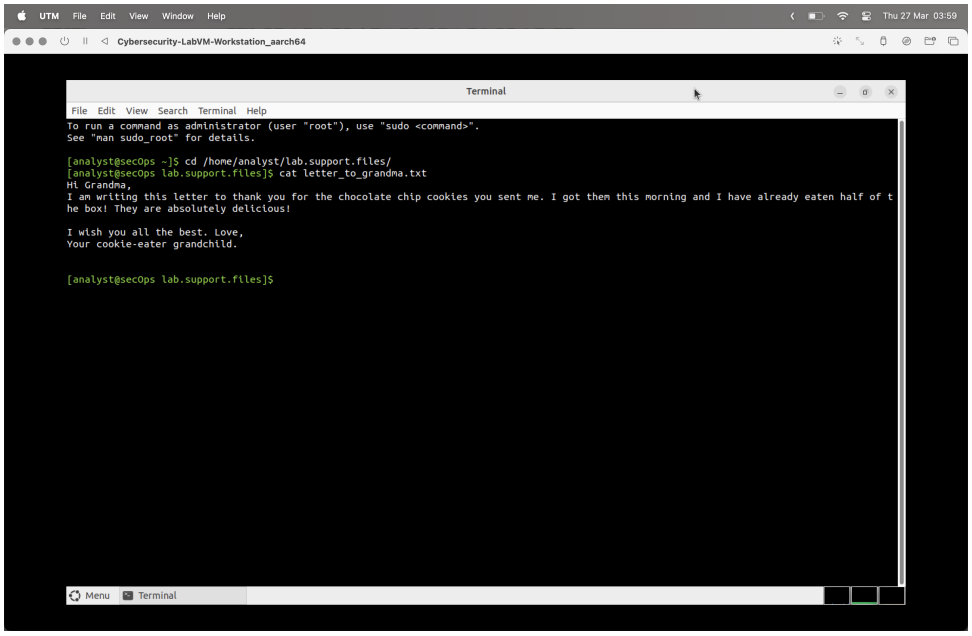

Figure 11: Contents of letter_to_grandma.txt

Then I calculated its SHA-256 hash:

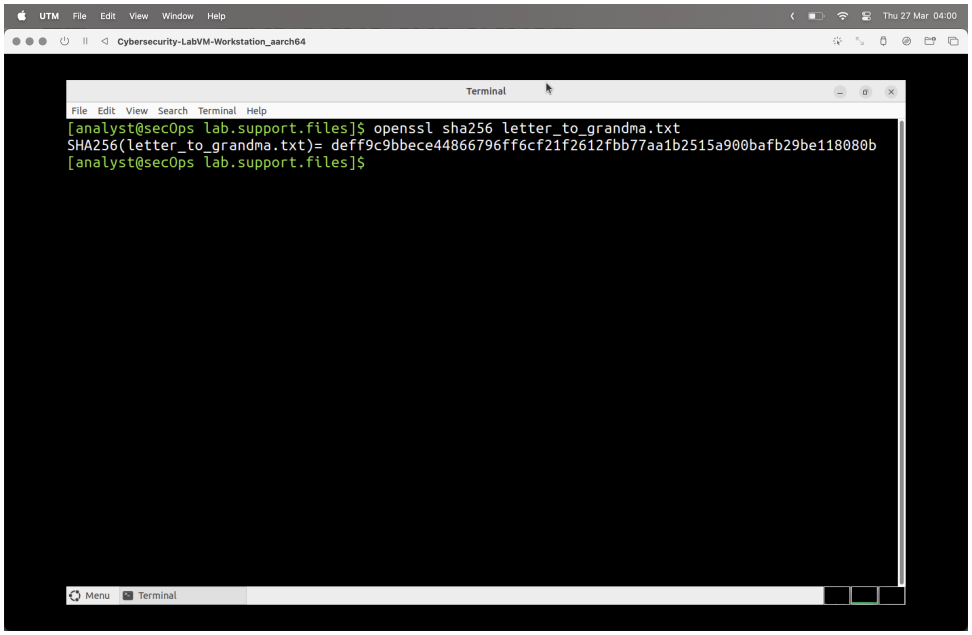

Figure 12: SHA-256 hash of original text file

The original SHA-256 hash was:
`deff9c9bbece44866796ff6cf21f2612fbb77aa1b2515a900bafb29be118080b`

3.1.2 Hash Comparison After File Modification

I modified the text file by changing "Hi Grandma" to "Hi Grandpa" and recalculated the hash:

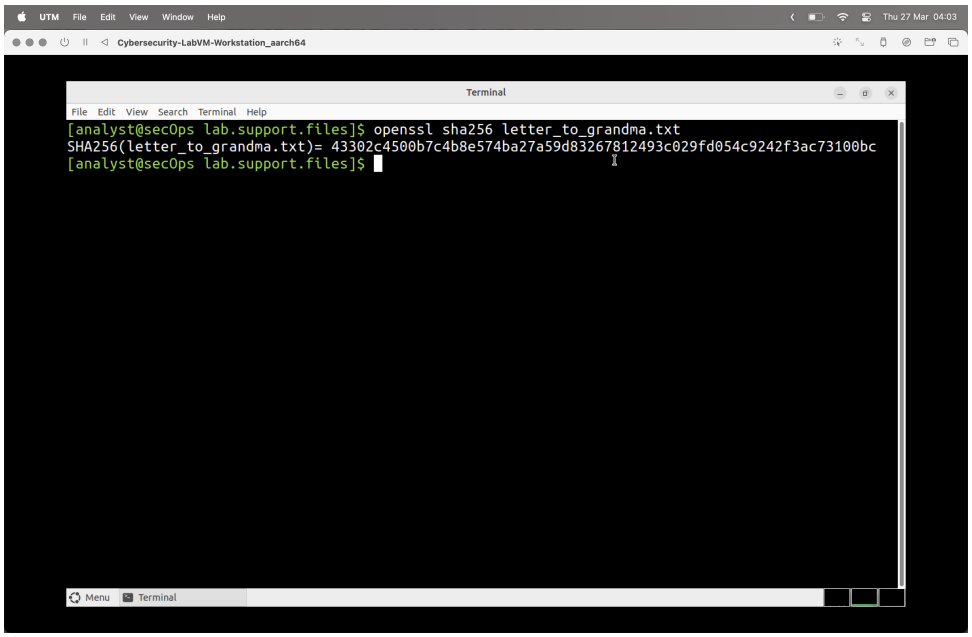

Figure 13: SHA-256 hash of modified text file

The new SHA-256 hash was:
`43302c4500b7c4b8e574ba27a59d83267812493c029fd054c9242f3ac73100bc`

Analysis of Hash Differences: The two hashes are completely different despite changing only a single character in the text file. This demonstrates the "avalanche effect" of cryptographic hash functions, where even a minor change to the input results in a completely different hash output. There isn't a pattern or partial match between the hashes - they appear entirely unrelated, which is a critical security property of strong hash functions.

3.1.3 Comparing Different Hash Algorithms

I also generated SHA-512 hashes and compared results from different utilities:

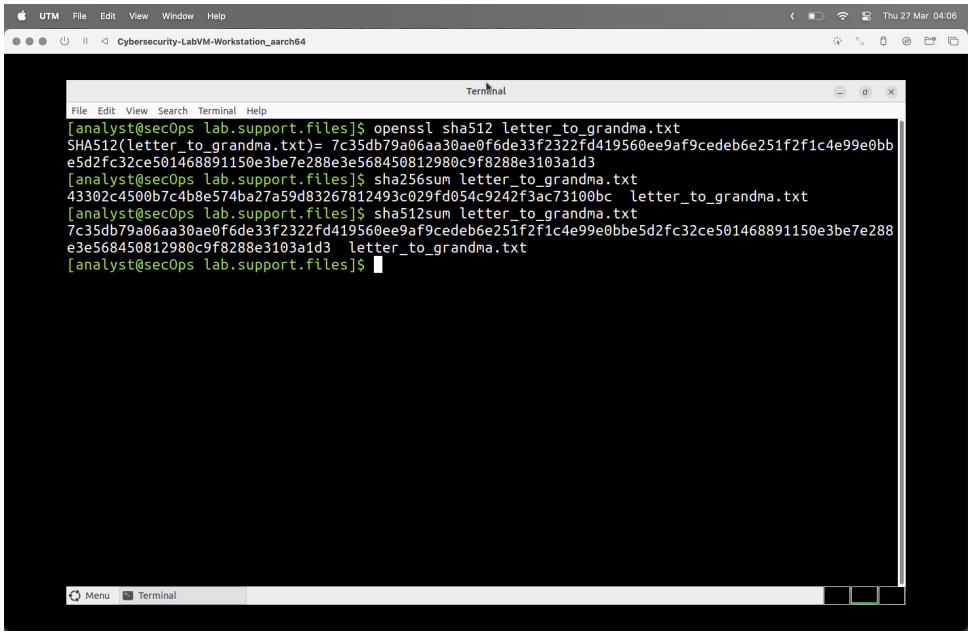

Figure 14: SHA-512, sha256sum, and sha512sum results

Comparison of Utilities: The hashes generated with **sha256sum** and **sha512sum** match exactly with those generated by OpenSSL's **openssl sha256** and **openssl sha512** commands. The only difference is in the output format - OpenSSL prefixes the hash with algorithm information and the filename, while the **sha256sum** and **sha512sum** utilities append the filename after the hash. This confirms that these tools implement the same standardized hash algorithms.

3.2 Part 2: Verifying Hashes

In this part, I verified the integrity of a downloaded file by comparing its hash with a reference hash.

3.2.1 Hash Verification Process

I first viewed the contents of the reference hash file:

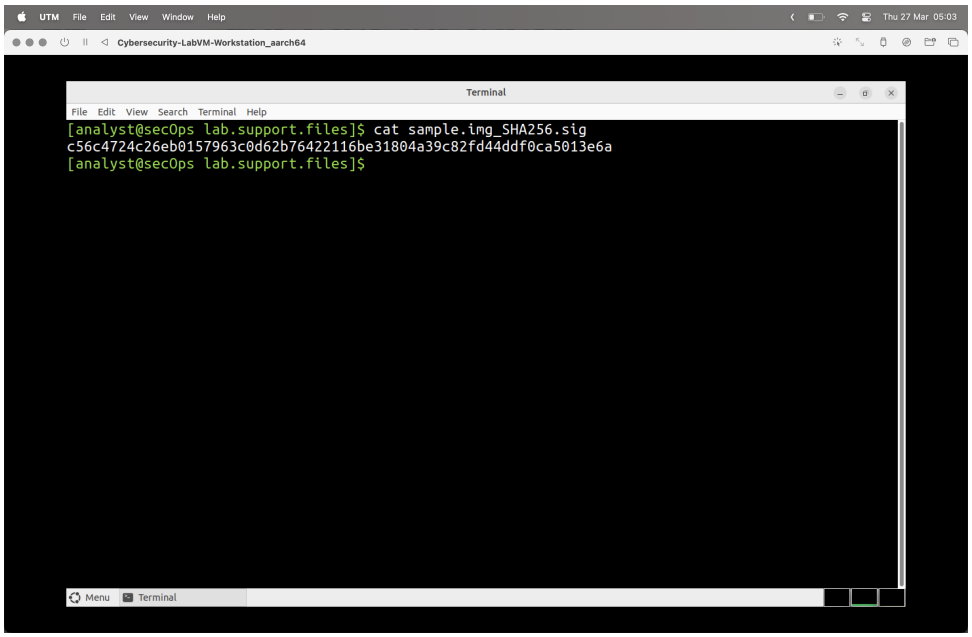

Figure 15: Contents of sample.img_SHA256.sig file

Then I calculated the SHA-256 hash of the sample file:

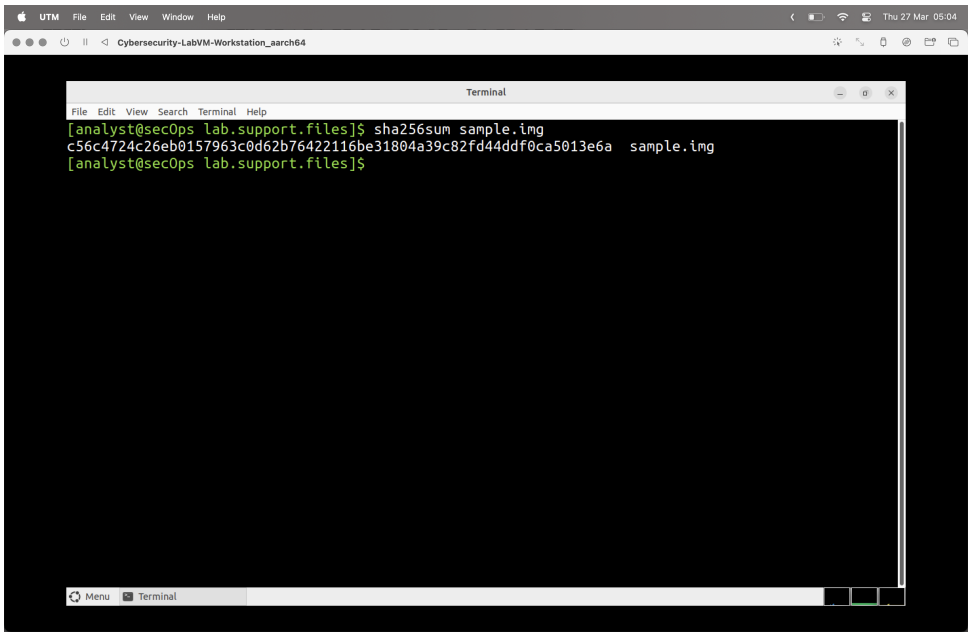

Figure 16: SHA-256 hash of sample.img file

Integrity Verification Results: The SHA-256 hash of sample.img calculated using sha256sum is: `c56c4724c26eb0157963c0d62b76422116be31804a39c82fd44ddf0ca5013e6a`

This hash exactly matches the reference hash in sample.img_SHA256.sig, confirming that the sample.img file was downloaded without errors or tampering. This verification process demonstrates how hash functions can be used to verify file integrity after transmission or storage.

3.3 Lab 3 Conclusion

Through this lab, I gained practical experience with cryptographic hash functions and their applications:

- I observed how hash functions produce completely different outputs even with minimal input changes
- I compared different hash algorithms (SHA-256 vs. SHA-512) and utilities
- I applied hashing to verify file integrity, demonstrating a real-world security application

This practical understanding of hashing is essential for various security applications, including:

- Data integrity verification
- Password storage (when combined with salting)
- Digital signatures
- File and message authentication